

```

!pip install pandas
!pip install numpy

import numpy as np

import numpy as np
np.__version__

!pip install matplotlib

import matplotlib as mat

mat.__version__

```

M1: Data Preprocessing

```

# Import libraries
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv("kc_house_data.csv")

# Display basic info
print(df.head())

```

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
waterfront \						
0	221900.0	3	1.00	1180	5650	1.0
0						
1	538000.0	3	2.25	2570	7242	2.0
0						
2	180000.0	2	1.00	770	10000	1.0
0						
3	604000.0	4	3.00	1960	5000	1.0
0						
4	510000.0	3	2.00	1680	8080	1.0
0						

	view	condition	grade	sqft_above	sqft_basement	yr_built
yr_renovated \						
0	0	3	7	1180	0	1955
0						
1	0	3	7	2170	400	1951
1991						
2	0	3	6	770	0	1933
0						
3	0	5	7	1050	910	1965
0						
4	0	3	8	1680	0	1987

0

	zipcode	lat	long	sqft_living15	sqft_lot15
0	98178	47.5112	-122.257	1340	5650
1	98125	47.7210	-122.319	1690	7639
2	98028	47.7379	-122.233	2720	8062
3	98136	47.5208	-122.393	1360	5000
4	98074	47.6168	-122.045	1800	7503

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 21613 entries, 0 to 21612
```

```
Data columns (total 21 columns):
```

#	Column	Non-Null Count	Dtype
0	id	21613 non-null	int64
1	date	21613 non-null	object
2	price	21613 non-null	float64
3	bedrooms	21613 non-null	int64
4	bathrooms	21613 non-null	float64
5	sqft_living	21613 non-null	int64
6	sqft_lot	21613 non-null	int64
7	floors	21613 non-null	float64
8	waterfront	21613 non-null	int64
9	view	21613 non-null	int64
10	condition	21613 non-null	int64
11	grade	21613 non-null	int64
12	sqft_above	21613 non-null	int64
13	sqft_basement	21613 non-null	int64
14	yr_built	21613 non-null	int64
15	yr_renovated	21613 non-null	int64
16	zipcode	21613 non-null	int64
17	lat	21613 non-null	float64
18	long	21613 non-null	float64
19	sqft_living15	21613 non-null	int64
20	sqft_lot15	21613 non-null	int64

```
dtypes: float64(5), int64(15), object(1)
```

```
memory usage: 3.5+ MB
```

```
None
```

```
# Check missing values
```

```
print(df.isnull().sum())
```

id	0
date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0

```

sqft_lot      0
floors        0
waterfront    0
view          0
condition     0
grade         0
sqft_above    0
sqft_basement 0
yr_built      0
yr_renovated  0
zipcode       0
lat           0
long          0
sqft_living15 0
sqft_lot15    0
dtype: int64

# Drop unnecessary columns
df = df.drop(['id', 'date'], axis=1)

# Check duplicates
print("Duplicates:", df.duplicated().sum())

Duplicates: 5

# Remove duplicates
df = df.drop_duplicates()

```

M2: Descriptive Statistics

```

# Summary statistics
print(df.describe())

```

	price	bedrooms	bathrooms	sqft_living
count	2.160800e+04	21608.000000	21608.000000	21608.000000
mean	5.400984e+05	3.370927	2.114738	2079.958950
std	3.671646e+05	0.930103	0.770238	918.505805
min	7.500000e+04	0.000000	0.000000	290.000000
25%	3.218375e+05	3.000000	1.750000	1429.250000
50%	4.500000e+05	3.000000	2.250000	1910.000000
75%	6.450000e+05	4.000000	2.500000	2550.000000
max	7.700000e+06	33.000000	8.000000	13540.000000

1.651359e+06

	floors	waterfront	view	condition
grade \				
count	21608.000000	21608.000000	21608.000000	21608.000000
21608.000000				
mean	1.494146	0.007544	0.234219	3.409478
7.656655				
std	0.539910	0.086527	0.766169	0.650782
1.175484				
min	1.000000	0.000000	0.000000	1.000000
1.000000				
25%	1.000000	0.000000	0.000000	3.000000
7.000000				
50%	1.500000	0.000000	0.000000	3.000000
7.000000				
75%	2.000000	0.000000	0.000000	4.000000
8.000000				
max	3.500000	1.000000	4.000000	5.000000
13.000000				

	sqft_above	sqft_basement	yr_built	yr_renovated
zipcode \				
count	21608.000000	21608.000000	21608.000000	21608.000000
21608.000000				
mean	1788.414846	291.544104	1970.999907	84.330155
98077.942614				
std	828.141155	442.612560	29.371463	401.517026
53.506883				
min	290.000000	0.000000	1900.000000	0.000000
98001.000000				
25%	1190.000000	0.000000	1951.000000	0.000000
98033.000000				
50%	1560.000000	0.000000	1975.000000	0.000000
98065.000000				
75%	2210.000000	560.000000	1997.000000	0.000000
98118.000000				
max	9410.000000	4820.000000	2015.000000	2015.000000
98199.000000				

	lat	long	sqft_living15	sqft_lot15
count	21608.000000	21608.000000	21608.000000	21608.000000
mean	47.560050	-122.213894	1986.583164	12770.108340
std	0.138578	0.140830	685.421474	27307.006892
min	47.155900	-122.519000	399.000000	651.000000
25%	47.470875	-122.328000	1490.000000	5100.000000
50%	47.571800	-122.230500	1840.000000	7620.000000
75%	47.678000	-122.125000	2360.000000	10083.250000
max	47.777600	-121.315000	6210.000000	871200.000000

```
# Mean house price
print("Mean Price:", df['price'].mean())
```

Mean Price: 540098.3690299889

```
# Median
print("Median Price:", df['price'].median())
```

Median Price: 450000.0

```
# Correlation
corr = df.corr()
#print(corr['price'].sort_values(ascending=False))
```

corr

	price	bedrooms	bathrooms	sqft_living	sqft_lot
floors \					
price	1.000000	0.308326	0.525146	0.702035	0.089652
0.256893					
bedrooms	0.308326	1.000000	0.515956	0.576640	0.031671
0.175608					
bathrooms	0.525146	0.515956	1.000000	0.754717	0.087751
0.500754					
sqft_living	0.702035	0.576640	0.754717	1.000000	0.172809
0.354181					
sqft_lot	0.089652	0.031671	0.087751	0.172809	1.000000
0.005113					
floors	0.256893	0.175608	0.500754	0.354181	-0.005113
1.000000					
waterfront	0.266371	-0.006591	0.063747	0.103817	0.021598
0.023731					
view	0.397409	0.079411	0.187892	0.284627	0.074744
0.029383					
condition	0.036345	0.028370	-0.124941	-0.058825	-0.008988
0.263728					
grade	0.667541	0.357143	0.665029	0.762893	0.113688
0.458074					
sqft_above	0.605559	0.477569	0.685376	0.876593	0.183508
0.524117					
sqft_basement	0.323839	0.303093	0.283824	0.435061	0.015263
0.245646					
yr_built	0.054062	0.154410	0.506038	0.318253	0.053154
0.489244					
yr_renovated	0.126489	0.018678	0.050880	0.055299	0.007671
0.006193					
zipcode	-0.053179	-0.152683	-0.203837	-0.199396	-0.129595
0.059153					
lat	0.307002	-0.008970	0.024587	0.052514	-0.085683
0.049595					
long	0.021587	0.129395	0.223065	0.240150	0.229530

0.125592
sqft_living15 0.585378 0.391571 0.568692 0.756398 0.144596
0.280080
sqft_lot15 0.082435 0.029197 0.087192 0.183258 0.718552 -
0.011180

	waterfront	view	condition	grade	
sqft_above \					
price	0.266371	0.397409	0.036345	0.667541	0.605559
bedrooms	-0.006591	0.079411	0.028370	0.357143	0.477569
bathrooms	0.063747	0.187892	-0.124941	0.665029	0.685376
sqft_living	0.103817	0.284627	-0.058825	0.762893	0.876593
sqft_lot	0.021598	0.074744	-0.008988	0.113688	0.183508
floors	0.023731	0.029383	-0.263728	0.458074	0.524117
waterfront	1.000000	0.401992	0.016648	0.082799	0.072076
view	0.401992	1.000000	0.045819	0.251426	0.167642
condition	0.016648	0.045819	1.000000	-0.144599	-0.158273
grade	0.082799	0.251426	-0.144599	1.000000	0.756090
sqft_above	0.072076	0.167642	-0.158273	0.756090	1.000000
sqft_basement	0.080584	0.276993	0.174061	0.168483	-0.051932
yr_built	-0.026150	-0.053196	-0.361306	0.446887	0.424084
yr_renovated	0.092949	0.103203	-0.060875	0.014396	0.023217
zipcode	0.030283	0.084857	0.002998	-0.184875	-0.261132
lat	-0.014272	0.006047	-0.014975	0.114100	-0.000837
long	-0.041916	-0.078598	-0.106590	0.198496	0.343739
sqft_living15	0.086466	0.280391	-0.092920	0.713396	0.731846
sqft_lot15	0.030698	0.072566	-0.003449	0.119319	0.194035

	sqft_basement	yr_built	yr_renovated	zipcode
lat \				
price	0.323839	0.054062	0.126489	-0.053179
0.307002				

bedrooms	0.303093	0.154410	0.018678	-0.152683	-
0.008970					
bathrooms	0.283824	0.506038	0.050880	-0.203837	
0.024587					
sqft_living	0.435061	0.318253	0.055299	-0.199396	
0.052514					
sqft_lot	0.015263	0.053154	0.007671	-0.129595	-
0.085683					
floors	-0.245646	0.489244	0.006193	-0.059153	
0.049595					
waterfront	0.080584	-0.026150	0.092949	0.030283	-
0.014272					
view	0.276993	-0.053196	0.103203	0.084857	
0.006047					
condition	0.174061	-0.361306	-0.060875	0.002998	-
0.014975					
grade	0.168483	0.446887	0.014396	-0.184875	
0.114100					
sqft_above	-0.051932	0.424084	0.023217	-0.261132	-
0.000837					
sqft_basement	1.000000	-0.133038	0.071315	0.074802	
0.110543					
yr_built	-0.133038	1.000000	-0.224722	-0.346923	-
0.148106					
yr_renovated	0.071315	-0.224722	1.000000	0.064402	
0.029270					
zipcode	0.074802	-0.346923	0.064402	1.000000	
0.267071					
lat	0.110543	-0.148106	0.029270	0.267071	
1.000000					
long	-0.144789	0.409606	-0.068622	-0.564047	-
0.135557					
sqft_living15	0.200367	0.326468	-0.002861	-0.278997	
0.048829					
sqft_lot15	0.017250	0.071059	0.007828	-0.147237	-
0.086426					

	long	sqft_living15	sqft_lot15
price	0.021587	0.585378	0.082435
bedrooms	0.129395	0.391571	0.029197
bathrooms	0.223065	0.568692	0.087192
sqft_living	0.240150	0.756398	0.183258
sqft_lot	0.229530	0.144596	0.718552
floors	0.125592	0.280080	-0.011180
waterfront	-0.041916	0.086466	0.030698
view	-0.078598	0.280391	0.072566
condition	-0.106590	-0.092920	-0.003449
grade	0.198496	0.713396	0.119319
sqft_above	0.343739	0.731846	0.194035

sqft_basement	-0.144789	0.200367	0.017250
yr_built	0.409606	0.326468	0.071059
yr_renovated	-0.068622	-0.002861	0.007828
zipcode	-0.564047	-0.278997	-0.147237
lat	-0.135557	0.048829	-0.086426
long	1.000000	0.334525	0.254448
sqft_living15	0.334525	1.000000	0.183165
sqft_lot15	0.254448	0.183165	1.000000

M3: Probability Concepts

```
# Average price based on waterfront
group_mean = df.groupby('waterfront')['price'].mean()

print(group_mean)

waterfront
0    5.315719e+05
1    1.661876e+06
Name: price, dtype: float64
```

import seaborn as sns import matplotlib.pyplot as plt

sns.boxplot(x='waterfront', y='price', data=df) plt.title("Price Comparison: NON Waterfront vs Waterfront") plt.show()

```
# Total number of houses
total = len(df)

# Houses with price > 500000
expensive = len(df[df['price'] > 500000])

# Probability
prob = expensive / total

print("Probability of expensive house:", prob)

Probability of expensive house: 0.41878008145131435
```

M4: Statistical Inference

```
# Average price based on waterfront
group_mean = df.groupby('waterfront')['price'].mean()

print(group_mean)

group_mean[1] ,group_mean[0]

(np.float64(1661876.0245398772), np.float64(531571.9172767545))
```



```
if group_mean[1] > group_mean[0]:
    print("Waterfront houses are more expensive")
else:
    print("No clear difference")
```

Waterfront houses are more expensive

M5: Regression

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
```

Features and target

```
X = df.drop('price', axis=1)
y = df['price']
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Predictions

```
y_pred = model.predict(X_test)
```

Evaluation

```
print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

MAE: 128203.50296474094

R2 Score: 0.7026188047056415

M6: Reporting & Visualization

Actual vs Predicted

```
plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted")
plt.show()
```

